KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

함수 3
lambda
map, filter

짧은 함수는 이름 없이 만들고
리스트 전체에 한 번에 쓰기

```
prices = [4500, 5000, 6500]

list(map(lambda p: p + 500, prices))
# [5000, 5500, 7000]

list(filter(lambda p: p >= 5000, prices))
# [5000, 6500]
```

오늘의 순서

1 왜 필요한가

되풀이하는 for와 append, map과 filter 그림, 가장 짧은 map

4–6쪽

2 함수를 값처럼, lambda

함수 담기와 넘기기, def와 lambda 비교, 바로 넘기기

7–11쪽

3 map

for 네 줄을 한 줄로, 하나씩 넣는 그림, map object, str과 len 넘기기

12–15쪽

4 filter

골라 담기를 한 줄로, True만 남는 그림, 딕셔너리 키, map과 이어 쓰기

16–19쪽

5 오류와 함정

번호와 len, 한 번만 꺼내지는 map, 괄호, return, 긴 lambda

20–24쪽

6 실습

10% 인상 가격표, 품절 아닌 메뉴, 튜플 주문 기록

25–27쪽

3쪽은 노트북 준비, 28–29쪽은 막힐 때와 한 장 요약이에요.

들어가기

20에서 이어서 해요

- 리스트, 튜플, for, append 13
- 딕셔너리 키로 for 돌기 15
- if, 한 줄 if else 17
- def, return, 괄호를 뺀 hello 19
- 키워드 인자 reverse=True 20

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

가격을 바꿀 때마다 for 네 줄을 다시 적어요

[1]
```python
prices = [4500, 5000, 6500]
up = []
for p in prices:
    up.append(p + 500)          # 500원 올리기
print(up)
down = []
for p in prices:
    down.append(p - 500)        # 500원 내리기
print(down)
```

```
[5000, 5500, 7000]
[4000, 4500, 6000]
```

달라진 건 한 줄뿐

p + 500과 p - 500만 달라요. 빈 리스트, for, append는 똑같이 되풀이해요.

자주 나오는 틀이에요

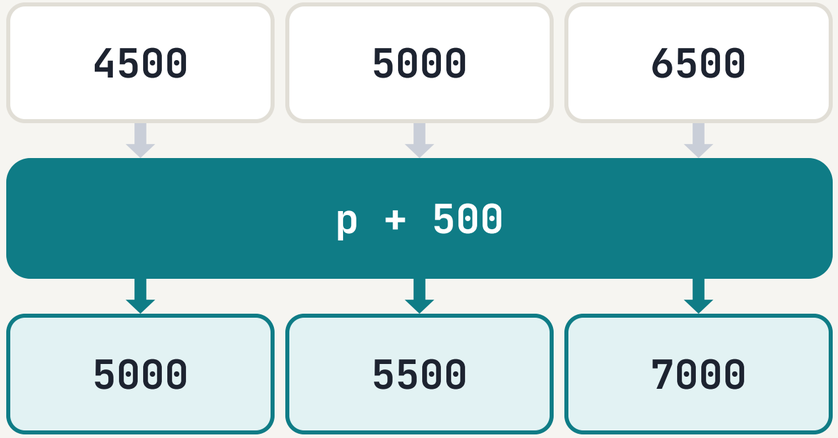
값마다 바꾸기, 조건에 맞는 값만 고르기는 카페 데이터에서 계속 나와요.

리스트 전체에 같은 일을 할 때마다 같은 틀을 다시 적고 있어요.

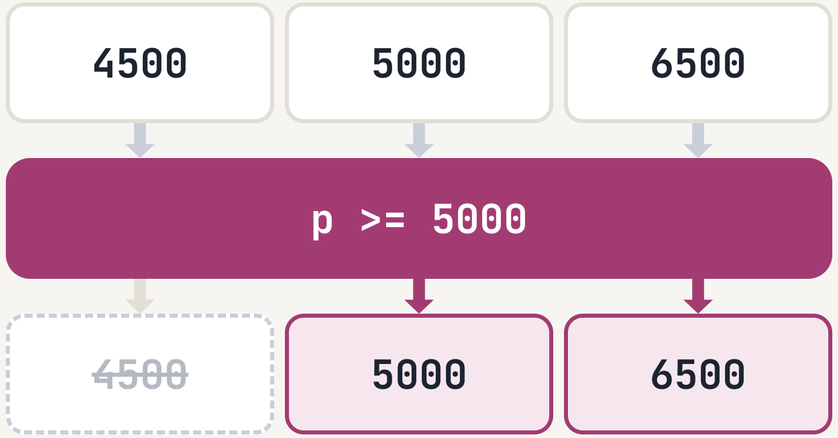
1. 왜 필요한가

map은 하나씩 바꾸고, filter는 골라내요

map: 값마다 바꿔요



filter: 조건에 맞는 값만 남겨요



map(함수, 리스트)

리스트의 값을 하나씩 함수에 넣고, 돌려받은 값을 모아요. 개수는 그대로예요.

filter(함수, 리스트)

함수가 True를 돌려준 값만 남겨요. 값은 그대로이고 개수가 줄어요.

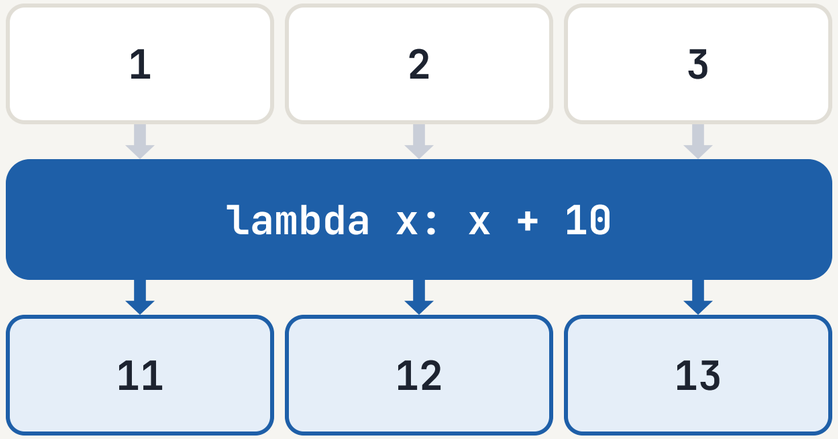
가운데 띠에 넣는 짧은 함수를 lambda로 만들어요.

1. 왜 필요한가

가장 짧은 map: 값마다 10 더하기

[3]
```python
numbers = [1, 2, 3]
print(list(map(lambda x: x + 10, numbers)))
```

```
[11, 12, 13]
```



`lambda x: x + 10` x를 받아 x + 10을 돌려주는 함수

`map(…, numbers)` numbers의 값마다 함수를 써요

`list(…)` 나온 값을 리스트로 모아요

함수와 리스트를 map에 넘기면 값마다 함수를 써서 결과를 모아 줘요.

2. 함수를 값처럼, lambda

함수도 변수에 담을 수 있어요

[1]
```python
def add10(x):
    return x + 10

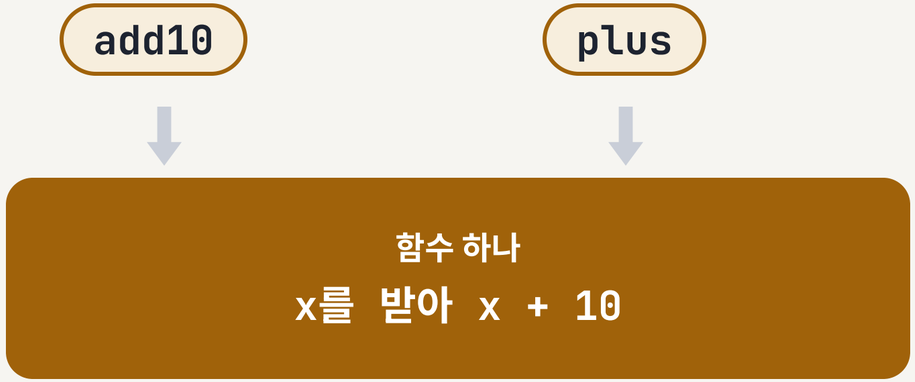
plus = add10
print(plus(5))
print(add10)
```
괄호 없이

Python

```
15
<function add10 at 0x7fdb817ce4b0>
```

19에서 본 그 모양이에요

괄호를 빼면 실행하지 않고 함수 자체가 나왔어요. 0x로 시작하는 숫자는 실행할 때마다 달라요.



이름표가 하나 더 붙어요

plus = add10은 함수를 실행하지 않아요. 같은 함수에 plus라는 이름을 하나 더 붙여서 plus(5)도 15가 돼요.

괄호를 빼면 함수를 실행하지 않고 값처럼 다룰 수 있어요.

2. 함수를 값처럼, lambda

함수를 다른 함수에 넘길 수 있어요

[2]
```python
def double(x):
    return x * 2

def apply(f, x):
    return f(x)

print(apply(add10, 5))
print(apply(double, 5))
```
받은 함수를 불러요

Python

```
15
10
```

`apply(add10, 5)` → `add10(5)` → `15`

`apply(double, 5)` → `double(5)` → `10`

함수를 받는 함수

apply는 f 칸에 함수를 받아서 f(x)로 불러요. 넘길 때는 괄호 없이 이름만 적어요. map과 filter도 이렇게 함수를 받아요.

함수도 인자로 넘길 수 있어요. 넘길 때는 이름만 적어요.

2. 함수를 값처럼, lambda

lambda는 한 줄로 만드는 이름 없는 함수예요

def로 만들기

```
def add10(x):
    return x + 10
```

lambda로 만들기

```
lambda x: x + 10
```

`lambda`	함수를 만든다는 표시예요. def 대신 써요
`:`	콜론 뒤가 돌려줄 값이에요

`x`	매개변수예요. 괄호 없이 적어요
`x + 10`	return 없이도 이 값을 돌려줘요

이름, 괄호, return을 빼고 돌려줄 값 하나만 남긴 모양이에요.

2. 함수를 값처럼, lambda

lambda는 다른 함수에 바로 넘길 때 써요

변수에 담아 부르기

[3]
```python
add10 = lambda x: x + 10
print(add10(5))
print(add10)
```

```
15
<function <lambda> at 0x7fdb817cf270>
```

이름 없이 바로 넘기기

[4]
```python
print(apply(lambda x: x * 3, 5))
```

```
15
```

여러 번 부를 함수는 def로

add10 = lambda보다 def add10(x):가 읽기 쉬워요.

이름이 <lambda>예요

lambda 함수에는 이름이 없어요. add10은 그 함수를 담은 변수일 뿐이에요.

한 번 쓸 함수는 그 자리에

앞 장의 apply에 lambda를 바로 넘겼어요. def로 이름을 붙이지 않아도 돼요.

한 번 쓰고 말 짧은 함수는 lambda로 그 자리에 적어요.

2. 함수를 값처럼, lambda

매개변수 여러 개, 한 줄 if else도 돼요

[5]

```python
price_of = lambda price, qty: price * qty          쉼표로 둘
print(price_of(4500, 2))
label = lambda p: "비쌈" if p >= 5000 else "보통"   17의 한 줄 if
print(label(6500), label(4500))
```

```
9000
비쌈 보통
```

if 문은 못 넣어요

콜론 뒤에는 값 하나만 와요. 한 줄 if else는 값이라서 돼요.

	def	lambda
이름	있어요	없어요
몸통	여러 줄	값 하나
return	적어요	안 적어요
언제	여러 번 부를 함수	다른 함수에 바로 넘길 때

3. map

for 네 줄이 map 한 줄이 돼요

[1]
```python
prices = [4500, 5000, 6500]

def add500(p):
    return p + 500

print(list(map(add500, prices)))
print(list(map(lambda p: p + 500, prices)))
print(prices)
```

def 이름 넘기기

lambda 바로 넘기기

원본은 그대로

Python

```
[5000, 5500, 7000]
[5000, 5500, 7000]
[4500, 5000, 6500]
```

1파트의 for 같은 결과

```python
up = []
for p in prices:
    up.append(p + 500)
```

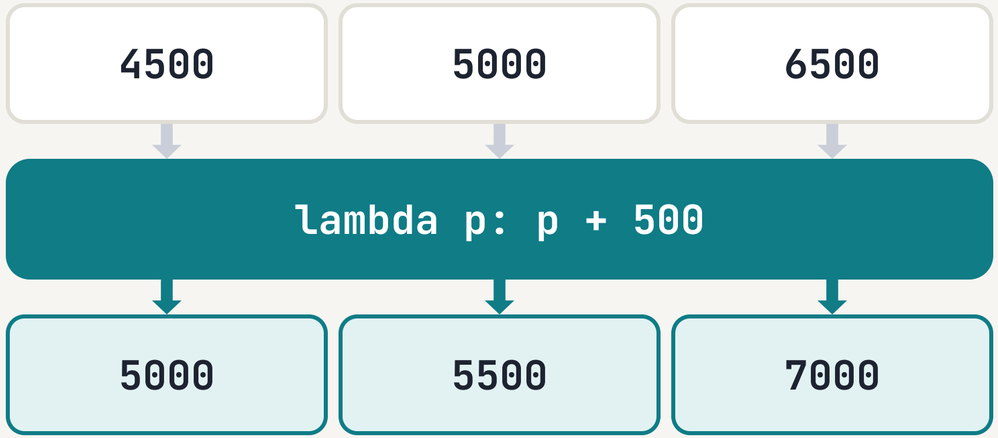
map은 새 결과를 만들어요

세 방법 모두 [5000, 5500, 7000]이에요. prices는 바뀌지 않아요. 13의 sorted처럼 원본을 두고 새 결과를 줘요.

map(함수, 리스트)는 for와 append로 새 리스트를 만드는 일을 한 줄로 줄여요.

3. map

map은 값을 하나씩 함수에 넣어요



1	`p = 4500`	4500 + 500을 돌려줘요
2	`p = 5000`	5000 + 500을 돌려줘요
3	`p = 6500`	6500 + 500을 돌려줘요

순서도 개수도 그대로

넣은 순서대로 결과가 나와요. 값이 셋이면 결과도 셋이에요.

p는 lambda의 매개변수

값이 하나씩 p 칸에 들어가요. 13의 for p in prices:의 p와 같은 역할이에요.

map이 리스트의 값을 차례로 lambda에 넣고, 돌려받은 값을 모아요.

3. map

map만 찍으면 <map object>가 나와요

[2]
```python
m = map(lambda p: p + 500, prices)
print(m)
print(list(m))
```

```
<map object at 0x7fe9b3214840>
[5000, 5500, 7000]
```

[3]
```python
for p in map(lambda p: p + 500, prices):
    print(p)
```

```
5000
5500
7000
```

0x로 시작하는 숫자는 화면마다 달라요

map은 꺼내 줄 준비만 해요

map object는 달라고 하면 값을 하나씩 꺼내 주는 묶음이에요. 아직 결과를 다 만들지 않았어요.

list()가 끝까지 꺼내요

list로 감싸면 값을 끝까지 꺼내 리스트로 모아요. for도 하나씩 꺼내 쓸 수 있어요.

결과를 리스트로 보려면 list()로 감싸요.

3. map

이미 있는 함수도 이름만 넘겨요

[4]
```python
print(list(map(str, prices)))
print(", ".join(map(str, prices)))
print(list(map(int, ["3", "1", "2"])))
print(list(map(len, ["coffee", "latte", "tea"])))
```

```
['4500', '5000', '6500']
4500, 5000, 6500
[3, 1, 2]
[6, 5, 3]
```

join은 글자만 이어요

13의 join에 숫자 리스트를 넣으면 오류가 나요. map(str, prices)로 글자로 바꿔 넘기면 한 줄로 이어 찍을 수 있어요.

`str`	숫자를 글자로. join 앞에
`int`	글자를 숫자로. 12의 타입 바꾸기
`len`	값마다 길이를 재요

str, int, len처럼 이미 있는 함수로 되는 일은 lambda 없이 이름만 넘겨요.

4. filter

for와 if로 골라 담던 일을 filter 한 줄로

```
[1]  prices = [4500, 5000, 6500, 3000]
     big = []
     for p in prices:                                  17의 골라 담기
         if p >= 5000:
             big.append(p)
     print(big)
     print(list(filter(lambda p: p >= 5000, prices)))   filter 한 줄
                                                                  Python
```

```
[5000, 6500]
[5000, 6500]
```

filter(함수, 리스트)

함수가 True를 돌려준 값만 남겨요. lambda 자리에는 True나 False가 나오는 조건을 적어요.

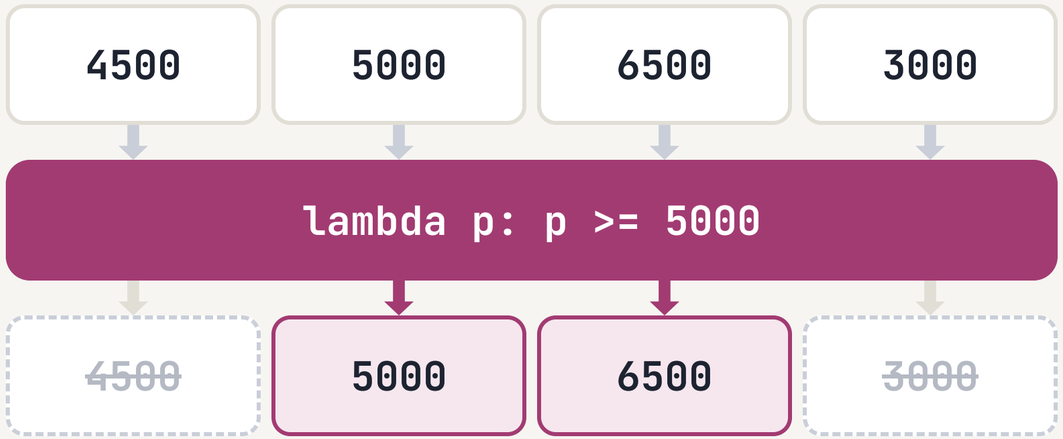
원본은 그대로

filter도 새 결과를 줘요. 결과를 보려면 map처럼 list로 감싸요.

for 안의 if로 골라 담는 일은 filter 한 줄로 줄여요.

4. filter

True를 받은 값만 아래로 내려가요



5000도 남아요

>=는 같아도 True예요. 12의 비교 연산과 같아요.

[2]
```python
check = lambda p: p >= 5000
print(check(4500), check(5000))
print(check(6500), check(3000))
```

```
False True
True False
```

p	p >= 5000	filter는
4500	False	버려요
5000	True	남겨요
6500	True	남겨요
3000	False	버려요

filter에 넘기는 함수는 True나 False를 돌려주는 조건이에요.

4. filter

딕셔너리를 넘기면 키가 하나씩 들어가요

[3]
```python
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 0}
print(list(filter(lambda m: stock[m] > 0, stock)))
```

```
['coffee', 'cake']
```

coffee	latte	cake	tea
3	0	2	0

재고가 0보다 큰 키만 → `['coffee', 'cake']`

m에는 키가 들어가요

15에서 for m in stock:이 키를 꺼낸 것과 같아요. 값은 stock[m]으로 봐요.

결과는 키 리스트

품절이 아닌 메뉴 이름만 남았어요. 재고 숫자는 들어 있지 않아요.

딕셔너리를 넘기면 키로 골라서, 조건에 맞는 키만 남겨요.

4. filter

고른 다음 바꾸려면 filter 결과를 map에 넘겨요

[4]

```python
big = list(filter(lambda p: p >= 5000, prices))
print(big)
print(list(map(lambda p: p * 9 // 10, big)))
print(list(map(lambda p: p * 9 // 10, filter(lambda p: p >= 5000, prices))))
```

두 번에 나눠 쓰기

한 줄에 겹쳐 쓰기

Python

```
[5000, 6500]
[4500, 5850]
[4500, 5850]
```

나눠 쓰면 읽기 쉬워요

5000원 이상만 고르고, 그 결과를 10% 할인해요. 중간 결과 big을 찍어 확인할 수 있어요.

겹치면 안쪽부터 읽어요

filter가 먼저, map이 나중이에요. 괄호가 겹쳐서 읽기 어려워요. 23에서 더 짧게 쓰는 법을 배워요.

고르고 바꾸기를 같이 할 땐 filter로 고른 결과를 map에 넘겨요.

5. 오류와 함정

map 결과에는 번호도 len도 못 써요

[1]
```python
doubled = map(lambda x: x * 2, [1, 2, 3])
print(doubled[0])
```

```
TypeError: 'map' object is not subscriptable
```

[2]
```python
print(len(doubled))
```

```
TypeError: object of type 'map' has no len()
```

map object는 리스트가 아니에요

값을 하나씩 꺼내 줄 뿐이라 [0]으로 꺼내거나 len으로 셀 수 없어요. filter도 같아요.

[3]
```python
doubled = list(map(lambda x: x * 2, [1, 2, 3]))
print(doubled[0])
print(len(doubled))
```

```
2
3
```

번호로 꺼내거나 길이를 재려면 먼저 list()로 바꿔요.

map 결과는 한 번 꺼내면 비어요

[4]
```
m = map(lambda p: p + 500, [4500, 5000])
print(list(m))
print(list(m))
```
남은 값이 없어요 Python

```
[5000, 5500]
[]
```

첫 번째 list(m) → 5000 5500

두 번째 list(m) → [] 다 꺼낸 뒤라 비었어요

오류 없이 비어 있어요

첫 list(m)이 값을 끝까지 꺼냈어요. 같은 m에서 다시 꺼낼 값이 없어요.

[5]
```
m = map(lambda p: p + 500, [4500, 5000])
ups = list(m)
print(ups)
print(ups)
```
Python

```
[5000, 5500]
[5000, 5500]
```

두 번 이상 쓸 결과는 list()로 바꿔 변수에 담아 둬요.

5. 오류와 함정

넘길 함수에 괄호를 붙이면 바로 실행돼요

[6]
```python
def add10(x):
    return x + 10

print(list(map(add10(), [1, 2, 3])))
```
add10() Python

```
TypeError: add10() missing 1 required positional argument: 'x'
```

map보다 add10()이 먼저 불려요

괄호를 붙이면 그 자리에서 x 없이 실행해서 오류가 나요.

missing 1 ... 'x'

19에서 본 오류예요. 칸 x에 넣을 값이 없다는 뜻이에요.

`map(add10(), ...)` → `map(add10, ...)`

5. 오류와 함정

lambda 안에 return을 쓰면 SyntaxError가 나요

[7]
```
add10 = lambda x: return x + 10
```
return Python

```
SyntaxError: invalid syntax
```

콜론 뒤에는 값만 와요

return은 값이 아니라 문장이에요. lambda는 콜론 뒤의 값을 알아서 돌려줘요.

if 문, for 문, =도 못 넣어요

lambda 몸통은 값 하나예요. 줄을 나눠 적어야 하는 일은 def로 만들어요.

이렇게 고쳐요 return 빼기

```
add10 = lambda x: x + 10
```

오류 끝 괄호는 뺐어요

SyntaxError 끝의 (1234.py, line 1)은 셀마다 달라서 슬라이드에서 뺐어요.

lambda는 콜론 뒤에 돌려줄 값 하나만 적어요.

5. 오류와 함정

lambda가 길어지면 def로 바꿔요

lambda 한 줄

[8]
```python
label = lambda p: "비쌈" if p >= 6000 else ("보통" if p >= 4500 else "쌈")
print(label(6500), label(5000), label(3000))
```

```
비쌈 보통 쌈
```

def로 나눠 쓰기

[9]
```python
def label(p):
    if p >= 6000:
        return "비쌈"
    elif p >= 4500:
        return "보통"
    return "쌈"

print(label(6500), label(5000), label(3000))
```

```
비쌈 보통 쌈
```

결과는 같아요

조건이 두 개를 넘으면 lambda는 한눈에 안 읽혀요. def로 나누고 map(label, prices)처럼 이름을 넘겨요.

6. 실습

메뉴 가격을 10% 올린 표를 만들어요

[1]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print(list(map(lambda m: menu[m] * 11 // 10, menu)))        # 새 가격만

def line(m):                                                # 길면 def
    new = menu[m] * 11 // 10
    return f"{m:<7}{menu[m]:>6,}원 → {new:>6,}원"

print("\n".join(map(line, menu)))                           # 줄마다 이어 찍기
```

```
[4950, 5500, 7150]
coffee   4,500원 →  4,950원
latte    5,000원 →  5,500원
cake     6,500원 →  7,150원
```

m은 메뉴 이름

딕셔너리를 넘겨서 키가 들어가요. 가격은 menu[m]이에요.

\n으로 이어요

13의 join에 줄바꿈 \n을 넣어 한 줄에 하나씩 찍어요.

한 줄 계산은 lambda, 줄 만들기처럼 긴 일은 def로 만들어 map에 넘겨요.

6. 실습

품절이 아니고 5000원 미만인 메뉴를 골라요

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 5}
print(list(filter(lambda m: stock[m] > 0, menu)))
print(list(filter(lambda m: stock[m] > 0 and menu[m] < 5000, menu)))
```
Python

```
['coffee', 'cake', 'tea']
['coffee', 'tea']
```

m	stock[m] > 0	menu[m] < 5000	둘 다
coffee	True	True	True
latte	False	False	False
cake	True	False	False
tea	True	True	True

and로 이어요

17에서 배운 and예요. 둘 다 True일 때만 남아요.

조건이 두 개면 lambda 안에서 and로 이어요.

6. 실습

튜플로 적은 주문 기록을 고르고 꺼내요

[3]
```python
orders = [("kim", "latte", 2), ("lee", "cake", 1), ("park", "coffee", 3), ("kim", "tea", 1)]
many = list(filter(lambda o: o[2] >= 2, orders))
print(many)
print(list(map(lambda o: o[0], many)))
print(sum(map(lambda o: o[2], orders)))
```

Python

```
[('kim', 'latte', 2), ('park', 'coffee', 3)]
['kim', 'park']
7
```

`o[2] >= 2` 2잔 이상 산 주문만 골라요

`o[0]` 고른 주문에서 이름만 꺼내요

`sum(map(...))` 잔 수만 꺼내 바로 더해요

튜플 하나가 lambda의 o 칸에 들어가고, 번호로 칸을 꺼내요.

정리

막히면 오류의 마지막 줄을 봐요

1	**<map object at 0x...>가 나와요** map 결과를 그대로 찍었어요	`list(map(...))`
2	**`TypeError: 'map' object is not subscriptable`** map 결과에 번호를 붙였어요	**list()로 바꾸기**
3	**`TypeError: object of type 'map' has no len()`** map 결과에 len을 썼어요	**list()로 바꾸기**
4	**두 번째 list(m)이 []예요** map은 한 번 꺼내면 비어요	**리스트로 담아 두기**
5	**`TypeError: add10() missing 1 required ...`** 넘길 함수에 괄호를 붙였어요	`map(add10, xs)`
6	**`SyntaxError: invalid syntax`** lambda 안에 return을 적었어요	`lambda x: x + 10`
7	**lambda가 한눈에 안 읽혀요** 조건이 여러 개예요	**def로 만들어 넘기기**

결과가 이상하면 list()로 바꿔 중간 결과를 찍어 봐요.

한 장 요약

`lambda x: x + 10`

이름 없는 한 줄 함수

`plus = add10`

괄호 없이 함수 담기

`map(f, xs)`

값마다 f를 써요

`list(map(...))`

리스트로 모으기

`map(str, xs)`

있는 함수는 이름만

`filter(f, xs)`

True인 값만 남겨요

`filter(f, d)`

딕셔너리는 키가 들어가요

`def label(p):`

길어지면 def로 넘기기

함수를 넘길 땐 괄호 없이 이름만 적어요

다음 챕터: sorted, max, min의 key에 lambda 넣기